problem Set 7 - Answers

Problem 1 (Graph play)

1. Directed graph and its transposed graph

Example directed graph G

Edges: A -> B
B -> C
C -> D

Adjacency list:
G:
A: B
B: C
C: D
D: -

Transpose graph G^T (all edges reversed)

Edges:
B -> A
C -> B
D -> C

Adjacency list:
G^T:
A: -
B: A
C: B
D: C

Explanation:
The transpose of a directed graph is obtained by reversing ~~ver. ve~~ every edge.

2. Undirected graph and its inverse graph

Example undirected graph:

Edges:	Adjacency list
A–B	A: B
B–C	B: A C
C–D	C: B D
	D: C

inverse graph (complement graph):

Edges exist only if they were not in the original graph.

possible edges in inverse:

A – C
A – D
B – D

Adjacency list:

A: C D
B: D
C: A
D: A B

Explanation:

The inverse graph connects vertices that were not connected before.

3. what happens if the original graph is dense?

if the original graph is dense (many edges), then the inverse graph becomes sparse (few edges).

Reason:

most vertex pairs are already connected, so the inverse graph has only a few missing edges.

4. Example of an undirected graph and its dual graph

Example planar graph (square)

A - B
| |
D - C

Faces: outer face
inter face

Dual graph : Each face becomes a vertex.

so dual graph:

$F_1 - F_2$

Explanation:

Every edge between two faces becomes an edge in the dual graph.

5. why the dual is only defined for planar graphs?

A dual graph requires faces in a planar drawing.

only planar graphs can be drawn without edge crossings.

wich clearly define faces.

Example of non-planar graph:

$K_{3,3}$

Vertices: $\{A, B, C\}$
$\{D, E, F\}$

Edges connect every vertex from the first set to the second set.

Reason it has no dual:
$K_{3,3}$ cannot be drawn on a plane without edges crossing.
Because faces cannot be clearly defined, a dual graph cannot be constructed.

Problem 2 (Bron - kerbosch Algorithm)
Graph:
Vertices $V = \{A, B, C, D\}$
Edges AB, AC, BC, CD
Adjacency list: A: BC
B: AC
C: ABD
D: C

1. Initial call
Bron - kerbosch algorithm starts with:
$R = \{\}$
$P = \{A, B, C, D\}$
$X = \{\}$
Meaning:
R = current clique, P = possible vertices to add
X = already processed vertices

2. First two recursive calls

Step 1 (choose A)

$R = \{A\}$

$P = \{B, C\}$

$X = \{\}$

Neighbors of A -> B, C

Step 2 (choose B)

$R = \{A, B\}$

$P = \{C\}$

$X = \{\}$

Neighbors of B -> A, C

Step 3 (choose C)

$R = \{A, B, C\}$

$P = \{\}$

$X = \{\}$

Since P and X are empty, this is a maximal clique.

Reported clique:

$\{A, B, C\}$

Another branch

Start again from C.

$R = \{C\}$

$P = \{D\}$

$X = \{A, B\}$

Next step:

$R = \{C, D\}$

$P = \{\}$

$X = \{\}$

Another maximal clique:

$\{C, D\}$

3. All maximal cliques

Maximal cliques found:

$\{A, B, C\}$

$\{C, D\}$

Maximum clique

Maximum clique = clique with the most vertices.

Sizes: $\{A, B, C\} \rightarrow$ ~~siz~~ size 3

$\{C, D\} \rightarrow$ size 2

So the maximun clique is

$\{A, B, C\}$